LIMITBEYOND

FITNESS & GYM MANAGEMENT PLATFORM

Data Model & Entity Relationships

Version 1.0 · 74 entities across 11 domains

Document ID	**LB-DATA-1.0**
Status	**Draft for review**
Date	**16 September 2026**
Prepared by	**Bibek Jha**
Implements	**LB-SRS-1.1 · LB-ARCH-1.0 · LB-ENG-1.0**
Target	**PostgreSQL 16 with Row-Level Security**
Migrations	**Flyway, forward-only**

> **What this document fixes.** Every table, every column that matters, every relationship and its cardinality, and the constraints that encode a requirement in the database rather than in application code. Where a design choice was available, the reasoning is stated — because the expensive mistakes in a schema are the ones made silently and discovered after there is production data sitting on top of them.

Table of Contents

1. Conventions and Design Principles
2. Domain Map and Cross-Domain Relationships
3. Domain 1 — Platform and Tenancy
4. Domain 2 — Identity and Authorisation
5. Domain 3 — People and Profiles
   Including the universal fitness profile — the schema expression of FR-PFP-01
6. Domain 4 — Billing and Memberships
   Including gapless GST receipt numbering
7. Domain 5 — Attendance
8. Domain 6 — Scheduling, Classes and Bookings
   Including database-enforced capacity
9. Domain 7 — Training: Exercises, Workouts, Nutrition
10. Domain 8 — Progress and Goals
11. Domain 9 — Engagement: Feedback, Notifications, WhatsApp
12. Domain 10 — Insights and AI
13. Domain 11 — Public Surface, SEO and Leads
14. Cross-Cutting Tables
15. Complete Relationship Matrix
16. Row-Level Security Application
17. Indexing Strategy
18. Migration Sequence by Phase
19. Enum Catalogue
20. Appendix A — Deviations from Earlier Documents
21. Appendix B — Amendments Required to SRS and Architecture

1. Conventions and Design Principles

1.1 Naming

Object	Convention	Example
Table	`snake_case`, plural	`check_ins`, `pt_packages`
Column	`snake_case`, singular	`occurred_at`, `credits_remaining`
Primary key	Always `id`	`id uuid` (UUIDv7)
Foreign key	`<referenced_singular>_id`	`location_id`, `trainer_user_id`
Junction table	Both sides, alphabetical	`exercise_muscle_groups`
Boolean	`is_` / `has_` prefix	`is_published`, `has_waitlist`
Timestamp	`_at` suffix, always `timestamptz`	`created_at`, `revoked_at`
Date only	`_on` suffix, `date`	`valid_until_on`, `starts_on`
Money	`_minor` suffix, `bigint`	`amount_minor` (paise)
Index	`idx_<table>_<cols>`	`idx_check_ins_user_occurred`
Unique	`uq_<table>_<cols>`	`uq_gyms_slug`
Check	`ck_<table>_<rule>`	`ck_pt_packages_credits_non_negative`
RLS policy	`p_<table>_isolation`	`p_check_ins_isolation`

1.2 Principles

ID	Principle	Application
DM-01	Scope columns are mandatory	Every operational table carries `gym_id` and `location_id`, both `NOT NULL`. These are not conveniences — they are the columns RLS policies filter on, so a table missing them cannot be isolated.
DM-02	Invariants live in constraints	If a rule can be a `CHECK`, a `UNIQUE`, a partial unique index or a foreign key, it is one. Application code can be bypassed by the next code path; the database cannot.
DM-03	Money is never floating point	`bigint` minor units plus an explicit currency column. No `float`, no `double`, no `real` anywhere near an amount.
DM-04	Financial and factual history is append-only	Payments, attendance, credit movements and audit entries are never updated or deleted. Corrections are new rows referencing the original.
DM-05	Ledger plus balance, not balance alone	Where a running total exists (PT credits, class packs), an immutable transaction ledger is the truth and the balance column is a locked, constrained cache of it. The ledger explains *how* a balance got there.
DM-06	Soft delete for anything referenced	`deleted_at timestamptz` on entities that history points at. A hard `DELETE` of a member orphans their attendance and payments.
DM-07	Time is UTC, stored with zone	`timestamptz` throughout, rendered in the tenant's timezone at the edge. A `timestamp` without zone is a future bug in a multi-timezone product.
DM-08	Enums are Postgres types	Not free strings, not integer codes. Adding a value is a migration, which is correct — a new booking state should be a deliberate decision, not a typo someone inserted.
DM-09	`JSONB` only for genuinely variable shapes	Health screening answers, AI payloads, audit before/after, notification template variables. Never for data that is queried, joined or constrained.

ID	Principle	Application
DM-10	No triggers, no stored procedures	Business logic lives in the application where it is testable and reviewable. Constraints are declarations, not logic, and are the one exception.

1.3 Columns present on every operational table

```
id           uuid         PRIMARY KEY                -- UUIDv7: unguessable, time-ordered
gym_id       uuid         NOT NULL REFERENCES gyms(id)
location_id  uuid         NOT NULL REFERENCES locations(id)
created_at   timestamptz  NOT NULL DEFAULT now()
updated_at   timestamptz  NOT NULL DEFAULT now()
created_by   uuid         REFERENCES users(id)      -- null for system-generated rows
deleted_at   timestamptz                              -- present where DM-06 applies
```

Omitted from the entity cards that follow, to keep them readable. Assume all six unless a note says otherwise. Tables above the gym level — `gyms`, `users`, `subscription_offerings` — necessarily lack the scope columns and are called out where that applies.

Entity card legend: **PK** primary key **FK** foreign key **UQ** unique or part of a unique constraint bold border core entity amber holds sensitive personal data.

2. Domain Map and Cross-Domain Relationships

Seventy-four entities across eleven domains. The domains correspond to the ten code modules of LB-ARCH-1.0 §4.3, plus the public surface introduced by the SEO and directory decision (§13).

1 Platform 6 entities gyms, subscriptions, locations	**2 Identity** 7 entities users, role grants, audit	**3 People** 8 entities profiles, trainers, assignment	**4 Billing** 11 entities plans, payments, credits	**5 Attendance** 3 entities check-ins, QR secrets
6 Scheduling 8 entities timetable, sessions, bookings	**7 Training** 15 entities exercises, workouts, diet	**8 Progress** 3 entities metrics, records, goals	**9 Engagement** 9 entities feedback, notifications	**10 Insights** 2 entities AI generations, snapshots
11 Public & SEO 4 entities public pages, leads, redirects, content	**Cross-cutting** 4 entities job_queue, media_assets, idempotency_records, audit_log			

2.1 The three spine relationships

Almost everything in the schema hangs off three relationships. Getting these right is most of the design.

Relationship	Card.	Why it is shaped this way
`gyms` → `locations`	1 : N	A gym runs many locations. Every operational row carries both ids, not just `location_id`, even though location implies gym. The denormalisation is deliberate: RLS filters on `gym_id` first and a join to resolve it on every query would be ruinous.
`users` → `role_grants`	1 : N	Authority is a set of grants, never a column on the user. This is what allows Trainer + Location Admin simultaneously (`FR-AUTH-26`) and scoping per location (`FR-AUTH-28`). A `role` column on `users` could express neither.
`users` → `fitness_profiles`	1 : 1	Every account, whatever its role. Created with the user, never granted, never revoked (`FR-PFP-01`). This single relationship is what makes the dual-facet model real rather than aspirational.

> **The one relationship that does *not* exist, deliberately.** There is no `members` table and no `member_id` anywhere. A member is a user holding a `MEMBER` role grant — nothing more. Had "member" been its own entity, staff could not train without a second account, and every training table would need a nullable second owner column. Modelling membership as a grant rather than an identity is what makes `FR-PFP-01` fall out naturally instead of needing special cases throughout.

3. Domain 1 — Platform and Tenancy

The top of the hierarchy: the platform's gyms, their subscription lifecycle, and their physical locations. The two tables above gym scope — `gyms` and `subscription_offerings` — are the only operational tables without a `gym_id`, because one *is* the gym and the other is platform-wide catalogue.

gyms — tenant root

PK	id	uuid
UQ	slug	citext
	legal_name	text
	trading_name	text
	gstin	text
	state	gym_state
	timezone	text
	currency	char(3)
FK	logo_media_id	
	verified_at	tstz
	is_publicly_listed	

subscription_offerings — platform

PK	id	uuid
	name	text
	term_months	int
	price_minor	bigint
	max_locations	int
	max_members	int
	is_active	bool

gym_subscriptions

PK	id	uuid
FK	gym_id	
FK	offering_id	
	starts_on	date
	expires_on	date
	grace_until_on	date
	state	subscription_state

subscription_payments

PK	id	uuid
FK	subscription_id	
FK	gym_id	
	amount_minor	bigint
	method	payment_method
	reference	text
	received_on	date
FK	recorded_by_user_id	

locations

PK	id	uuid
FK	gym_id	
UQ	slug	citext
	name	text
	address_line1/2	text
	city, state, pincode	
	geo	geography(Point)
	checkin_radius_m	int
	floor_capacity	int
	is_publicly_listed	
	closed_at	tstz

location_hours

PK	id	uuid
FK	location_id	
UQ	day_of_week	int
	opens_at	time
	closes_at	time
	is_closed	bool

location_closures

PK	id	uuid
FK	location_id	
	closed_on	date
	reason	text

RELATIONSHIPS

From → To	Card.	On delete	Note
`gyms` → `locations`	1 : N	RESTRICT	A gym has at least one location; locations are closed, never deleted (`FR-ORG-14`).
`gyms` → `gym_subscriptions`	1 : N	RESTRICT	Successive terms. At most one in state `LIVE` — enforced by partial unique index.
`subscription_offerings` → `gym_subscriptions`	1 : N	RESTRICT	Offerings are retired, not deleted, so historical terms remain explicable.
`gym_subscriptions` → `subscription_payments`	1 : N	RESTRICT	Cash recorded by the Super Admin (`FR-SUB-02`). Append-only per `DM-04` .
`locations` → `location_hours`	1 : N	CASCADE	Exactly seven rows per location, one per weekday.
`locations` → `location_closures`	1 : N	CASCADE	Suppresses session generation (`FR-ORG-07`).

CONSTRAINTS THAT ENCODE A REQUIREMENT

```
-- FR-SUB-04: a gym has at most one live subscription at any time
CREATE UNIQUE INDEX uq_gym_subscriptions_live
    ON gym_subscriptions (gym_id) WHERE state = 'LIVE';

-- §13: the public URL is /gyms/{gym-slug}/{location-slug}, so location slugs
-- need only be unique within their gym, and gym slugs globally.
CREATE UNIQUE INDEX uq_locations_gym_slug ON locations (gym_id, slug);

-- Geofence radius must be sane: a 5km radius defeats the point (FR-ATT-02)
ALTER TABLE locations ADD CONSTRAINT ck_locations_radius
    CHECK (checkin_radius_m BETWEEN 20 AND 500);
```

`geography(Point,4326)` requires the PostGIS extension. It earns its place twice over: geofenced check-in needs an accurate distance calculation, and the public location pages need "gyms near me" ordering (§13). Computing great-circle distance in application code over a growing table would be both slower and wrong at the edges.

4. Domain 2 — Identity and Authorisation

The most consequential design in the schema. Authority is modelled as grants with explicit location scope, because the SRS requires that a trainer promoted to Location Admin at one site holds no authority at a sibling (`FR-AUTH-18` , `FR-AUTH-28`).

users — no gym scope

PK	id	uuid
UQ	phone_e164	text
UQ	email	citext
	password_hash	text
	first_name, last_name	
	date_of_birth	date
	gender	gender
FK	avatar_media_id	
	state	account_state
	phone_verified_at	
	deleted_at	

role_grants

PK	id	uuid
FK	user_id	
FK	gym_id	
UQ	role	role_type
	is_gym_wide	bool
FK	granted_by_user_id	
	granted_at	tstz
	revoked_at	tstz
	revoked_reason	text

role_grant_locations

PK	id	uuid
FK	role_grant_id	
FK	location_id	

refresh_tokens

PK	id	uuid
FK	user_id	
UQ	token_hash	text
	device_label	text
	expires_at	tstz
	revoked_at	tstz

user_consents

PK	id	uuid
FK	user_id	
	consent_type	consent_type
	version	text
	granted_at	tstz
	withdrawn_at	tstz

otp_attempts — rate limiting

PK	id	uuid
	identifier	text
	channel	otp_channel
	device_fingerprint	
	attempted_at	tstz
	was_successful	bool

audit_log — append-only

PK	id	uuid
FK	gym_id	nullable
FK	actor_user_id	
	action	text
	subject_type	text
	subject_id	uuid
	before	jsonb
	after	jsonb
	reason	text
	occurred_at	tstz

RELATIONSHIPS

From → To	Card.	On delete	Note
`users` → `role_grants`	1 : N	CASCADE	A user may hold several grants across gyms and roles.
`role_grants` → `role_grant_locations`	1 : N	CASCADE	Empty set when `is_gym_wide` ; otherwise the named locations.
`locations` → `role_grant_locations`	1 : N	RESTRICT	A location cannot be deleted while an admin is scoped to it.
`users` → `role_grants` (granted_by)	1 : N	SET NULL	The delegation chain (`FR-AUTH-25`): who appointed whom.
`users` → `refresh_tokens`	1 : N	CASCADE	One per device; revocable individually or wholesale (`FR-AUTH-13`).
`users` → `user_consents`	1 : N	RESTRICT	Never deleted — consent history is a DPDP obligation.

Why a junction table rather than a `location_ids uuid[]` column. LB-ARCH-1.0 §5.2 sketched the scope as an array. A junction table is the better model and supersedes that sketch: an array cannot have a foreign key, so it will eventually hold the id of a deleted location with nothing to stop it; "which admins cover location X" is an indexed join rather than a full scan of an array column; and `RESTRICT` on the location FK prevents closing a location out from under its administrators. The RLS context still needs a fast list, so the session filter builds the location CSV once per request from this table — one indexed query, not a per-row join. Recorded in Appendix A.

CONSTRAINTS

```sql
-- FR-AUTH-11: one live grant of a given role per user per gym.
-- Partial unique index, not a UNIQUE constraint: revoked grants stay as history
-- and must not block a later re-grant of the same role.
CREATE UNIQUE INDEX uq_role_grants_live
    ON role_grants (user_id, gym_id, role) WHERE revoked_at IS NULL;

-- FR-AUTH-18: only GYM_OWNER and SUPER_ADMIN may hold gym-wide scope.
ALTER TABLE role_grants ADD CONSTRAINT ck_role_grants_gym_wide_role
    CHECK (NOT is_gym_wide OR role IN ('GYM_OWNER','SUPER_ADMIN'));

-- FR-AUTH-22: last-admin protection. A partial unique index cannot express
-- "at least one", so this is enforced in the service layer under a lock on the
-- gym row — one of the few invariants the database genuinely cannot hold.
```

Where DM-02 does not reach. Last-admin protection is a *minimum cardinality* rule, and SQL constraints can only express maxima and shapes. It therefore lives in `IdentityService`, guarded by a row lock on the gym, with a dedicated concurrency test proving two simultaneous demotions cannot both succeed. Naming the exception matters: an invariant everyone assumes the database is holding, which it is not, is exactly how these fail.

5. Domain 3 — People and Profiles

This domain contains the schema's most important single decision. `fitness_profiles` is 1:1 with `users` — not with members, not with a role — which is `FR-PFP-01` made structural. No query in the training or progress domains needs to know what role its owner holds.

fitness_profiles — 1:1 users

PK	id	uuid
FK	user_id	UNIQUE
	height_cm	numeric(5,1)
	experience_level	level
	primary_goal	fitness_goal
	preferred_times	jsonb
	dietary_pref	dietary_preference

health_screenings — sensitive

PK	id	uuid
FK	user_id	
	answers	jsonb
	injuries	text
	conditions	text
	medications	text
	restrictions	text
	completed_at	tstz
	superseded_at	tstz

emergency_contacts

PK	id	uuid
FK	user_id	
	name, relationship	
	phone_e164	text

member_records — per gym

PK	id	uuid
FK	user_id	
FK	gym_id	
FK	primary_location_id	
	joined_on	date
	member_code	text
	source	member_source

member_location_access

PK	id	uuid
FK	member_record_id	
FK	location_id	
FK	granted_by_user_id	

trainer_profiles

PK	id	uuid
FK	user_id	
FK	gym_id	
	bio	text
	years_experience	int
	max_member_load	int
	pt_rate_minor	bigint
	is_publicly_listed	

trainer_specialisations

PK	id	uuid
FK	trainer_profile_id	
UQ	specialisation	enum

trainer_certifications

PK	id	uuid
FK	trainer_profile_id	
	name, issuer	text
	issued_on, expires_on	
FK	document_media_id	

trainer_availability

PK	id	uuid
FK	trainer_profile_id	
FK	location_id	
	day_of_week	int
	starts_at, ends_at	time

trainer_shifts

PK	id	uuid
FK	trainer_profile_id	
FK	location_id	
	starts_at, ends_at	tstz

trainer_assignments

PK	id	uuid
FK	member_user_id	
FK	trainer_user_id	
FK	gym_id	
	assigned_at, ended_at	
FK	assigned_by_user_id	

RELATIONSHIPS

From → To	Card.	On delete	Note
`users` → `fitness_profiles`	1 : 1	CASCADE	**Every user, unconditionally.** Created in the same transaction as the user (`FR-PFP-01`).
`users` → `health_screenings`	1 : N	CASCADE	Versioned: a new screening supersedes rather than overwrites, so a trainer can see what was true when a plan was written.
`users` → `member_records`	1 : N	CASCADE	One per gym the person is a member of. Holds the gym-specific facts a global user row must not.
`member_records` → `member_location_access`	1 : N	CASCADE	Explicit sibling-location grants only (`FR-ORG-05`) — never implicit.
`users` → `trainer_profiles`	1 : N	CASCADE	One per gym. A person may coach at more than one.
`trainer_profiles` → `trainer_specialisations`	1 : N	CASCADE	Drives class-type eligibility (`FR-CLS-01`) and public trainer listings.

From → To	Card.	On delete	Note
`trainer_profiles` → `trainer_availability`	1 : N	CASCADE	Weekly recurring windows, per location. Constrains PT scheduling (`FR-PT-02`).
`trainer_profiles` → `trainer_shifts`	1 : N	CASCADE	Dated roster. Attendance is evaluated against it (`FR-ATT-09`).
`users` ⇄ `users` via `trainer_assignments`	M : N	RESTRICT	Member ↔ trainer over time. History preserved via `ended_at` (`FR-MEM-05`).

```
-- One live trainer per member per gym (FR-MEM-05)
CREATE UNIQUE INDEX uq_trainer_assignments_live
    ON trainer_assignments (member_user_id, gym_id) WHERE ended_at IS NULL;

-- FR-PFP-08: nobody is their own trainer
ALTER TABLE trainer_assignments ADD CONSTRAINT ck_trainer_assignments_not_self
    CHECK (member_user_id <> trainer_user_id);

-- FR-PFP-07: staff may be coached. Deliberately NO constraint requiring the
-- member side to hold a MEMBER grant — adding one would break staff coaching,
-- which is exactly the mistake this schema is shaped to prevent.
```

6. Domain 4 — Billing and Memberships

Money. Every design choice here favours auditability over convenience: ledgers rather than mutable balances, allocations rather than a single payment-to-thing pointer, and gapless receipt numbering because GST requires it.

plans — gym-wide

PK	id uuid
FK	gym_id
	name text
	duration_months int
	price_minor bigint
	includes_group_classes
	is_active bool

plan_locations

PK	id uuid
FK	plan_id
FK	location_id

memberships

PK	id uuid
FK	member_record_id
FK	plan_id
	starts_on, expires_on
	price_agreed_minor
	discount_minor
	state membership_state

membership_freezes

PK	id uuid
FK	membership_id
	freeze_from, freeze_to
	days_extended int
	reason text

pt_packages

PK	id uuid
FK	member_user_id
FK	trainer_user_id
	credits_total int
	credits_remaining int
	price_minor bigint
	valid_until_on date

pt_credit_transactions

PK	id uuid
FK	pt_package_id
	delta int
	reason credit_reason
FK	pt_appointment_id
	occurred_at tstz

class_packs

PK	id uuid
FK	member_record_id
FK	class_type_id null
	entries_total int
	entries_remaining int
	valid_until_on date

payments — append-only

PK	id uuid
FK	member_record_id
	amount_minor bigint
	currency char(3)
	method payment_method
	reference text
	received_on date
FK	collected_by_user_id

payment_allocations

PK	id uuid
FK	payment_id
	target_type enum
	target_id uuid
	amount_minor bigint

receipts

PK	id uuid
FK	payment_id
UQ	receipt_number text
	financial_year text
	taxable_minor, cgst, sgst, igst
	hsn_sac text
FK	pdf_media_id

receipt_sequences

PK	id uuid
FK	gym_id
UQ	financial_year text
	last_number bigint

refunds

PK	id uuid
FK	payment_id
	amount_minor bigint
	reason text
FK	approved_by_user_id

RELATIONSHIPS

From → To	Card.	On delete	Note
`gyms` → `plans`	1 : N	RESTRICT	Gym-wide catalogue, Gym Owner only (`FR-ORG-12`).
`plans` ⇄ `locations`	M : N	CASCADE	A plan may be valid at some locations only.
`member_records` → `memberships`	1 : N	RESTRICT	Successive terms. At most one active — partial unique index.
`memberships` → `membership_freezes`	1 : N	CASCADE	Each freeze extends `expires_on` by `days_extended` (`FR-BIL-08`).
`pt_packages` → `pt_credit_transactions`	1 : N	RESTRICT	**The ledger is the truth** (`DM-05`); `credits_remaining` is its locked cache.
`payments` → `payment_allocations`	1 : N	RESTRICT	One payment may settle a membership and a PT package at once. This is what makes partial payment and dues computable (`FR-BIL-04`).

From → To	Card.	On delete	Note
`payments` → `receipts`	1 : 1	RESTRICT	Every recorded payment produces exactly one receipt (`FR-BIL-05`).
`payments` → `refunds`	1 : N	RESTRICT	Never mutate the original (`DM-04` , `FR-BIL-12`).

> **Gapless receipt numbering, and why a Postgres `SEQUENCE` will not do.** GST requires invoice numbers to be consecutive with no gaps. A `SEQUENCE` is explicitly non-transactional — it does not roll back — so an aborted transaction permanently burns a number and leaves a hole. The `receipt_sequences` table exists solely to provide a number that *is* transactional: increment it under a row lock in the same transaction as the receipt insert, and a rollback returns the number. Slower than a sequence, and correct, which is the trade that matters when a tax authority is the audience.

```sql
-- Gapless allocation, inside the receipt-creating transaction
UPDATE receipt_sequences
   SET last_number = last_number + 1
 WHERE gym_id = ? AND financial_year = ?
RETURNING last_number;                 -- row-locked; rolls back with the transaction

-- Only one active membership per member (FR-BIL-02)
CREATE UNIQUE INDEX uq_memberships_active
    ON memberships (member_record_id) WHERE state = 'ACTIVE';

-- FR-BIL-10: the balance cache can never go negative or exceed the total
ALTER TABLE pt_packages ADD CONSTRAINT ck_pt_packages_credits
    CHECK (credits_remaining >= 0 AND credits_remaining <= credits_total);

-- A payment may never be over-allocated across its targets
-- (sum of allocations <= payment amount) — enforced in service under lock,
-- since SQL cannot express a cross-row aggregate constraint.
```

7. Domain 5 — Attendance

Small domain, high write volume, and the one most exposed to offline replay. Its correctness rests entirely on the idempotency constraint.

check_ins — append-only

PK	id	uuid
FK	gym_id, location_id	
FK	user_id	
	occurred_at	tstz
	checked_out_at	tstz
	method	checkin_method
	purpose	checkin_purpose
FK	session_id	null
FK	recorded_by_user_id	
	is_deferred	bool
	distance_m	int
	is_flagged	bool
UQ	idempotency_key	text

location_qr_secrets

PK	id	uuid
FK	location_id	
	secret	bytea
	rotation_seconds	int
	rotated_at	tstz

attendance_streaks — derived

PK	id	uuid
FK	user_id	
FK	gym_id	
	current_streak	int
	longest_streak	int
	last_check_in_on	date
	habitual_hour	int

RELATIONSHIPS AND CONSTRAINTS

From → To	Card.	On delete	Note
`users` → `check_ins`	1 : N	RESTRICT	Never deleted. A member's attendance history outlives their membership.
`locations` → `check_ins`	1 : N	RESTRICT	
`sessions` → `check_ins`	1 : N	SET NULL	Set when the arrival is for a booked class (`FR-ATT-12`); null for a general gym visit.
`locations` → `location_qr_secrets`	1 : N	CASCADE	Rotating signing key. History retained briefly so a code scanned moments before rotation still validates.
`users` → `attendance_streaks`	1 : 1 per gym	CASCADE	Derived, recomputed by the `StreakRecalc` job. Safe to rebuild from `check_ins` at any time.

```sql
-- FR-ATT-05 + ADR-06: the single constraint that makes offline replay safe.
-- The client reuses one key across every retry, so a duplicate submission
-- collides here and the service returns the original row instead of inserting.
ALTER TABLE check_ins ADD CONSTRAINT uq_check_ins_idempotency
    UNIQUE (user_id, idempotency_key);

-- FR-ATT-13: attendance purpose keeps trainer utilisation honest.
-- Without it, a trainer's own workout inflates both staffing and footfall.
-- checkin_purpose: 'SHIFT' | 'PERSONAL_WORKOUT' | 'BOTH'
```

> **`habitual_hour` is not decoration.** It is the derived field that makes `FR-WA-06` possible — the WhatsApp nudge sent at the member's usual gym time on a day they have not come. Computed by the streak job as the modal check-in hour over a trailing window, it is stored rather than calculated on demand because the nudge job scans every member each hour and cannot afford an aggregate per user.

8. Domain 6 — Scheduling, Classes and Bookings

The most concurrency-sensitive domain, and the clearest justification for the single-transaction monolith of ADR-01. Capacity is enforced by the database, not by the application checking and hoping.

class_types

PK	id	uuid
FK	gym_id	
UQ	slug	citext
	name	text
	duration_minutes	int
	default_capacity	int
	intensity	intensity
	required_specialisation	
	is_publicly_listed	

rooms

PK	id	uuid
FK	location_id	
	name	text
	capacity	int

timetable_entries — recurrence rule

PK	id	uuid
FK	location_id	
FK	class_type_id	
FK	trainer_user_id	
FK	room_id	null
	day_of_week	int
	starts_at	time
	capacity	int
	delivery_mode	enum
	effective_from/to	date

sessions — occurrence

PK	id	uuid
FK	timetable_entry_id	null
FK	location_id, class_type_id	
FK	trainer_user_id	
	starts_at, ends_at	tstz
	capacity	int
	booked_count	int
	state	session_state
	delivery_mode	enum
	meeting_url	text
	meeting_passcode	
	recording_url	text

bookings

PK	id	uuid
FK	session_id	
FK	user_id	
	state	booking_state
	waitlist_position	int
	booked_at	tstz
	cancelled_at	tstz
	was_late_cancel	bool
FK	class_pack_id	null
UQ	idempotency_key	

pt_appointments

PK	id	uuid
FK	pt_package_id	
FK	member_user_id	
FK	trainer_user_id	
FK	location_id	
	starts_at, ends_at	tstz
	state	appointment_state
	notes	text
	completed_at	tstz

session_ratings

PK	id	uuid
FK	session_id	
FK	user_id	
	rating	int
	comment	text

RELATIONSHIPS

From → To	Card.	On delete	Note
`timetable_entries` → `sessions`	1 : N	SET NULL	Sessions are **materialised, not computed**. `FR-CLS-04` requires amending a single occurrence without disturbing the pattern, which a computed view could not express. `SET NULL` lets a generated session survive deletion of its rule.
`sessions` → `bookings`	1 : N	RESTRICT	Capacity-constrained. See the constraint below.
`class_types` → `sessions`	1 : N	RESTRICT	
`users` → `sessions` (trainer)	1 : N	RESTRICT	Substitution rewrites this pointer (`FR-CLS-12`).
`class_packs` → `bookings`	1 : N	SET NULL	Set when an entry was consumed; restored on session cancellation (`FR-CLS-13`).
`pt_packages` → `pt_appointments`	1 : N	RESTRICT	Completion writes a `-1` row to the credit ledger (`FR-PT-03`).
`sessions` → `session_ratings`	1 : N	CASCADE	One rating per attendee per session.

How capacity is actually enforced. `booked_count` is denormalised onto `sessions` specifically so that a `CHECK` constraint can police it. The booking transaction locks the session row, increments the counter, and inserts the booking; if the increment would breach capacity the constraint aborts the whole transaction. This is what makes `NFR-PRF-05` a database guarantee rather than an application intention — twenty concurrent requests for one remaining place produce exactly one booking, with no application-level coordination.

```sql
-- The constraint that cannot be argued with
ALTER TABLE sessions ADD CONSTRAINT ck_sessions_capacity
    CHECK (booked_count >= 0 AND booked_count <= capacity);

-- FR-CLS-06: one live booking per person per session. Partial unique index,
-- so a cancelled booking does not block re-booking the same session.
CREATE UNIQUE INDEX uq_bookings_live
    ON bookings (session_id, user_id)
 WHERE state IN ('BOOKED','WAITLISTED');

-- FR-CLS-14: no double-booked trainer or room. GiST exclusion over time ranges —
-- the database refuses overlap outright rather than the application checking for it.
ALTER TABLE sessions ADD CONSTRAINT ex_sessions_trainer_overlap
    EXCLUDE USING GIST (
        trainer_user_id WITH =,
        tstzrange(starts_at, ends_at) WITH &&
    ) WHERE (state <> 'CANCELLED');

-- FR-CLS-07: waitlist order is unique within a session
CREATE UNIQUE INDEX uq_bookings_waitlist_position
    ON bookings (session_id, waitlist_position) WHERE state = 'WAITLISTED';
```

The `EXCLUDE USING GIST` constraint requires the `btree_gist` extension. It is worth enabling for this one constraint alone: trainer double-booking is otherwise a check that must be remembered in every code path that creates or reschedules a session, and forgetting it produces a trainer expected in two rooms at once.

9. Domain 7 — Training: Exercises, Workouts and Nutrition

The largest domain, and the one that must never branch on role. Every table here is keyed by `user_id`, not by a member identifier — the schema-level expression of `FR-PFP-01`.

9.1 Exercise catalogue

muscle_groups

PK	id	uuid
UQ	slug	citext
	name	text
	body_region	enum

equipment

PK	id	uuid
UQ	slug	citext
	name	text

exercises

PK	id	uuid
FK	gym_id	null = global
UQ	slug	citext
	name	text
	discipline	enum
	difficulty	level
	instructions	text[]
FK	demo_media_id	
	is_publicly_listed	

exercise_muscle_groups

PK	id	uuid
FK	exercise_id	
FK	muscle_group_id	
	role	PRIMARY|SECONDARY

exercise_equipment

PK	id	uuid
FK	exercise_id	
FK	equipment_id	

> **`gym_id` is nullable on `exercises`, uniquely in this schema.** A null means a platform-curated global exercise visible to every gym; a value means a gym's own addition (`FR-EXR-03`). This is the single deliberate exception to `DM-01`, and it requires its RLS policy to admit nulls explicitly — `gym_id IS NULL OR gym_id = current_setting(...)`. Missing that clause is an easy mistake whose symptom is an empty exercise library, so it is called out in §16.

9.2 Workouts

workout_templates

PK	id	uuid
FK	gym_id	
FK	author_user_id	
	name	text
	estimated_minutes	int

template_exercises

PK	id	uuid
FK	workout_template_id	
FK	exercise_id	
	position	int
	rest_seconds	int
	notes	text

template_sets — prescribed

PK	id	uuid
FK	template_exercise_id	
	set_number	int
	target_reps	int
	target_weight_kg	numeric
	target_seconds	int

routines

PK	id	uuid
FK	gym_id	
FK	author_user_id	
	name	text
	weeks	int

routine_days

PK	id	uuid
FK	routine_id	
UQ	week_number	int
UQ	day_of_week	int
FK	workout_template_id	
	load_multiplier	numeric

routine_assignments

PK	id	uuid
FK	routine_id	
FK	user_id	
FK	assigned_by_user_id	
	starts_on, ends_on	date
	state	enum

workout_logs

PK	id	uuid
FK	user_id	
FK	routine_assignment_id	null
FK	workout_template_id	null
FK	location_id	
	performed_at	tstz
	duration_minutes	int
	is_deferred	bool
UQ	idempotency_key	

workout_log_sets

PK	id	uuid
FK	workout_log_id	
FK	exercise_id	
	set_number	int
	reps	int
	weight_kg	numeric(6,2)
	seconds	int
	rpe	numeric(3,1)

personal_records — derived

PK	id	uuid
FK	user_id	
FK	exercise_id	
UQ	record_type	enum
	value	numeric
FK	workout_log_set_id	
	achieved_at	tstz

9.3 Nutrition

diet_plans

PK	id	uuid
FK	user_id	
FK	author_user_id	
	objective	text
	starts_on, ends_on	
	state	enum

diet_plan_versions

PK	id	uuid
FK	diet_plan_id	
UQ	version_number	int
	change_reason	text
	kcal_target	int
	protein/carbs/fat_g	
FK	ai_generation_id	null
FK	approved_by_user_id	

diet_plan_meals

PK	id	uuid
FK	diet_plan_version_id	
	meal_type	enum
	time_of_day	time
	notes	text

diet_plan_items

PK	id	uuid
FK	diet_plan_meal_id	
FK	food_id	null
	free_text	text
	quantity	numeric
	unit	text

foods Phase 4

PK	id	uuid
UQ	slug	citext
	name	text
	portion_g	numeric
	kcal, protein, carbs, fat	
	region	text

diet_adherence_logs Phase 4

PK	id	uuid
FK	user_id	
FK	diet_plan_meal_id	
	logged_on	date
	adherence	enum

diet_consultations

PK	id	uuid
FK	user_id	
FK	gym_id	
	subject	text
	state	enum

consultation_messages

PK	id	uuid
FK	diet_consultation_id	
FK	author_user_id	
	body	text
FK	attachment_media_id	
	sent_at	tstz

KEY RELATIONSHIPS

From → To	Card.	On delete	Note
`exercises` ⇄ `muscle_groups`	M : N	CASCADE	Junction carries `role` — the same exercise is primary for one group and secondary for another, which a plain join table could not express.
`workout_templates` → `template_exercises` → `template_sets`	1:N:N	CASCADE	Prescription. Three levels because sets within an exercise vary independently.
`routines` → `routine_days`	1 : N	CASCADE	Unique on (routine, week, weekday). `load_multiplier` expresses progressive overload (`FR-WKT-10`) without duplicating templates per week.
`users` → `workout_logs`	1 : N	RESTRICT	**Keyed on user, not member.** Staff training uses the identical path.
`workout_logs` → `workout_log_sets`	1 : N	CASCADE	What was actually performed, against what was prescribed.
`diet_plans` → `diet_plan_versions`	1 : N	CASCADE	Versioning per `FR-NUT-04` : meals hang off the *version*, so revising preserves the prior plan intact.
`diet_plan_versions` → `diet_plan_meals` → `diet_plan_items`	1:N:N	CASCADE	
`foods` → `diet_plan_items`	1 : N	SET NULL	Nullable: Phase 2 plans use `free_text` ; Phase 4 links to the food database without migrating old plans.

```sql
-- FR-WKT-07: one record per user per exercise per type, replaced when beaten
CREATE UNIQUE INDEX uq_personal_records
    ON personal_records (user_id, exercise_id, record_type);

-- An item is either a catalogue food or free text — never neither
ALTER TABLE diet_plan_items ADD CONSTRAINT ck_diet_plan_items_has_content
    CHECK (food_id IS NOT NULL OR free_text IS NOT NULL);

-- FR-AI-03: an AI-drafted plan version cannot be visible without human approval
ALTER TABLE diet_plan_versions ADD CONSTRAINT ck_diet_versions_ai_approved
    CHECK (ai_generation_id IS NULL OR approved_by_user_id IS NOT NULL);
```

That last constraint is worth dwelling on. `FR-AI-03` says no AI-generated content reaches a member without recorded human approval. Expressed as a `CHECK`, it becomes impossible to violate — an AI-sourced plan version without an approver cannot be inserted at all. A requirement that would otherwise depend on every future code path remembering it is instead held by the database permanently. This is `DM-02` at its most valuable.

10. Domain 8 — Progress and Goals

Progress photographs are out of scope, so this domain reduces to numeric measurement over time — which is also the shape that charts and analytics actually want.

body_metrics

PK	id	uuid
FK	user_id	
UQ	metric_type	metric_type
UQ	recorded_on	date
	value	numeric(7,2)
	unit	text
FK	recorded_by_user_id	

goals

PK	id	uuid
FK	user_id	
	goal_type	enum
	metric_type	null
	target_value	numeric
	target_on	date
	state	enum

measurement_reminders

PK	id	uuid
FK	user_id	
	cadence_days	int
	last_prompted_on	date

> **Why one narrow table rather than a wide one.** The obvious alternative is a `progress_entries` table with a column per measurement — `weight_kg`, `chest_cm`, `waist_cm` and so on. The narrow `(metric_type, value)` shape wins on three counts: adding a measurement is a new enum value rather than a schema migration and a nullable column on every historical row; charting a single metric is an indexed filter rather than a column projection with nulls; and a member who records only weight does not create a row that is 90% null. The cost is that fetching every metric for one date is a pivot — done once in the query layer, and worth it.

```sql
-- One value per metric per day per person; re-entry updates rather than duplicates
CREATE UNIQUE INDEX uq_body_metrics_daily
    ON body_metrics (user_id, metric_type, recorded_on);

-- Reject physiologically impossible values — catches unit confusion (lb entered as kg)
ALTER TABLE body_metrics ADD CONSTRAINT ck_body_metrics_sane
    CHECK (value > 0 AND value < 1000);
```

11. Domain 9 — Engagement

feedback

PK	id	uuid
FK	gym_id, location_id	
FK	author_user_id	
	category	enum
	body	text
	state	enum
	is_anonymous	bool

feedback_responses

PK	id	uuid
FK	feedback_id	
FK	author_user_id	
	body	text

coaching_notes staff → member

PK	id	uuid
FK	recipient_user_id	
FK	author_user_id	
	note_type	enum
	body	text
	is_internal	bool

announcements

PK	id	uuid
FK	gym_id	
FK	location_id	null = all
	title, body	text
	published_at	tstz

notifications

PK	id	uuid
FK	recipient_user_id	
	event_type	enum
	channel	enum
	payload	jsonb
	delivery_state	enum
	read_at	tstz

notification_preferences

PK	id	uuid
FK	user_id	
UQ	category	enum
	via_push, via_whatsapp	
	quiet_from, quiet_to	time

device_tokens

PK	id	uuid
FK	user_id	
UQ	expo_push_token	text
	platform	enum
	revoked_at	tstz

whatsapp_messages

PK	id	uuid
FK	recipient_user_id	
	template_name	text
	variables	jsonb
	provider_message_id	
	delivery_state	enum
	sent_on	date

message_templates

PK	id	uuid
FK	gym_id	null = default
UQ	event_type, channel	
	body	text

```
-- FR-WA-10: per-member daily cap on outbound WhatsApp. The index makes the
-- "how many today" check cheap enough to run before every send.
CREATE INDEX idx_whatsapp_messages_daily_cap
    ON whatsapp_messages (recipient_user_id, sent_on);

-- FR-FBK-09: anonymous feedback still records its author for abuse handling,
-- but the flag governs disclosure. Never omit author_user_id to achieve anonymity —
-- an unattributable complaint cannot be investigated or followed up.
```

12. Domain 10 — Insights and AI

ai_generations — audit of every call

PK	id	uuid
FK	gym_id	
FK	subject_user_id	
FK	requested_by_user_id	
	feature	ai_feature
	model	text
	prompt_inputs	jsonb
	output	jsonb
	state	DRAFT|APPROVED|REJECTED
FK	approved_by_user_id	
	approved_at	tstz

report_snapshots — materialised

PK	id	uuid
FK	gym_id	
FK	location_id	null
	report_type	enum
	period_start, period_end	
	data	jsonb
	computed_at	tstz

`ai_generations` satisfies `FR-AI-06` — every generation logged with inputs, output, model and approver. It is also the table that makes provider replacement safe: because the model name is recorded per generation, a change of provider leaves history explicable rather than mysterious. `report_snapshots` backs `NFR-PRF-06`'s asynchronous escape for reports too heavy to compute inside a request.

13. Domain 11 — Public Surface, SEO and Leads

> **New in this document.** The public directory and lead capture were out of scope in LB-SRS-1.1. The decision to build a Next.js public surface on a single domain brings both back. The schema consequence is small and mostly consists of fields the entities should arguably have had anyway — a slug, a publish flag, and somewhere to put a meta description.

13.1 How SEO fields are carried

Not a polymorphic `seo_metadata` table. Slugs need per-entity unique constraints and foreign-key-backed lookups, and a generic key-value table gives neither. The fields live on the entities that have public pages:

Entity	Public URL	SEO columns
`gyms`	`/gyms/{gym-slug}`	`slug` (globally unique), `is_publicly_listed`, `meta_title`, `meta_description`, `og_media_id`
`locations`	`/gyms/{gym-slug}/{location-slug}`	`slug` (unique within gym), `is_publicly_listed`, `meta_title`, `meta_description`, plus address and `geo` already present
`class_types`	`/classes/{class-slug}`	`slug`, `is_publicly_listed`, `description`
`trainer_profiles`	`/gyms/{gym}/trainers/{user-slug}`	`is_publicly_listed`, `bio`
`exercises`	`/exercises/{exercise-slug}`	`slug`, `is_publicly_listed` — a genuinely large long-tail surface

13.2 New entities

leads — public write path

PK	id	uuid
FK	gym_id, location_id	
	name	text
	phone_e164	text
	email	citext
	interest	lead_interest
FK	class_type_id	null
	message	text
	source	lead_source
	state	lead_state
FK	assigned_to_user_id	
FK	converted_user_id	
	ip_hash	text
UQ	idempotency_key	

lead_activities

PK	id	uuid
FK	lead_id	
FK	actor_user_id	
	activity	enum
	note	text
	next_follow_up_on	

content_pages — blog / marketing

PK	id	uuid
UQ	slug	citext
	title, excerpt	text
	body_markdown	text
	meta_title/description	
FK	hero_media_id	
	published_at	tstz

url_redirects

PK	id	uuid
UQ	from_path	text
	to_path	text
	status_code	int

RELATIONSHIPS

From → To	Card.	On delete	Note
`locations` → `leads`	1 : N	RESTRICT	A lead always belongs to the location whose public page produced it.
`leads` → `lead_activities`	1 : N	CASCADE	The follow-up pipeline visible in the admin console.

From → To	Card.	On delete	Note
`leads` → `users` (converted)	N : 1	SET NULL	Set when a lead registers, closing the funnel loop and making conversion rate measurable.

> **`leads` is the only table in the schema written by unauthenticated traffic.** That makes it the platform's sole public attack surface and it needs treating as such: a per-IP and per-phone rate limit, a CAPTCHA or equivalent on the form, a honeypot field, server-side validation of the phone format, and a hashed IP stored for abuse investigation rather than the raw address. Note also that RLS cannot protect the insert — there is no authenticated context to filter on — so the `gym_id` and `location_id` are derived server-side from the page's slug, never accepted from the request body. A client-supplied `gym_id` here would let anyone inject leads into any gym's pipeline.

13.3 Structured data generated from these tables

Schema.org type	Built from	What it earns
`LocalBusiness` / `ExerciseGym`	`locations` + `location_hours`	Map pack eligibility, opening hours and address shown directly in results. The single highest-value markup for "gym near me".
`Schedule` / `Event`	`sessions` + `class_types`	Class times eligible for rich results — genuinely rare among gym sites, and a real differentiator.
`Person`	`trainer_profiles`	Trainer pages that can rank on name searches.
`AggregateRating`	`session_ratings`	Star ratings in results. Only where ratings are genuine — fabricating these is both a policy violation and a manual-action risk.
`Article`	`content_pages`	Blog eligibility.

> **Why the schema is the SEO strategy.** Every one of the markup types above is generated from data the platform already holds for operational reasons. Fifty gyms with three locations each is 150 `LocalBusiness` pages with real addresses, real hours and a live timetable — all on one domain, all compounding its authority. That is a content position most gym websites cannot reach, and it costs no writing. The blog is the only part that requires ongoing effort, which is precisely why it is the lowest priority of the four public surfaces.

14. Cross-Cutting Tables

media_assets	
PK	id uuid
FK	gym_id null
UQ	public_id text
	sensitivity PUBLIC|SENSITIVE
	delivery_type text
	content_type, bytes
	width, height int
FK	uploaded_by_user_id

job_queue	
PK	id uuid
	job_type text
	payload jsonb
	state job_state
	priority int
	run_after tstz
	attempts int
	last_error text

idempotency_records	
PK	id uuid
FK	user_id
UQ	idempotency_key text
	endpoint text
	response_status int
	response_body jsonb
	expires_at tstz

> **`media_assets.public_id` is the schema's enforcement of NN-4.** There is no URL column anywhere in this database — not here, not on `users`, not on `exercises`. Every media reference is a foreign key to this table, and every URL is constructed at read time from the `public_id` plus the sensitivity rules. Because there is nowhere to *put* a full Cloudinary URL, the rule cannot be broken by forgetting it. The `sensitivity` column is likewise not advisory: `MediaService` reads it to decide signed versus public delivery.

```sql
-- ADR-09: the whole job queue mechanism in one index
CREATE INDEX idx_job_queue_claimable
    ON job_queue (priority DESC, run_after)
 WHERE state = 'PENDING';                -- partial: pending rows are a tiny slice

-- Generic idempotency for endpoints without their own natural key.
-- check_ins, workout_logs, bookings and leads carry their own (see each domain);
-- this table serves everything else and expires after 24 hours.
CREATE UNIQUE INDEX uq_idempotency_records
    ON idempotency_records (user_id, idempotency_key);
```

15. Complete Relationship Matrix

Every relationship crossing a domain boundary. Within-domain relationships are in each domain's own section.

Parent	Child	Card.	On del.	Purpose
`gyms`	all gym-scoped tables	1 : N	RESTRICT	Tenant root. The `gym_id` every RLS policy filters on.
`locations`	all location-scoped tables	1 : N	RESTRICT	The isolation boundary of `CON-09`.
`users`	`fitness_profiles`	1 : 1	CASCADE	Universal, per `FR-PFP-01`.
`users`	`role_grants`	1 : N	CASCADE	All authority.
`users`	`check_ins`, `workout_logs`, `body_metrics`, `goals`, `diet_plans`, `bookings`	1 : N	RESTRICT	Personal training data — keyed on user, never on a member entity.
`users`	`member_records`, `trainer_profiles`	1 : N	CASCADE	Per-gym facets of one person.
`member_records`	`memberships`, `payments`, `class_packs`	1 : N	RESTRICT	Commercial relationship with one gym.
`sessions`	`bookings`, `check_ins`, `session_ratings`	1 : N	RESTRICT / SET NULL	Attendance links both to session and location.
`pt_packages`	`pt_appointments`, `pt_credit_transactions`	1 : N	RESTRICT	Credits consumed on completion.
`exercises`	`template_exercises`, `workout_log_sets`, `personal_records`	1 : N	RESTRICT	Catalogue referenced by prescription and by performance.
`media_assets`	`users`, `gyms`, `exercises`, `receipts`, `content_pages`, `consultation_messages`, `trainer_certifications`	1 : N	SET NULL	All media by reference. No URL is ever stored (`NN-4`).
`ai_generations`	`diet_plan_versions`	1 : N	SET NULL	Provenance, with approval enforced by `CHECK`.
`leads`	`users` (converted)	N : 1	SET NULL	Closes the public funnel.
`users`	`audit_log` (actor)	1 : N	RESTRICT	Never deleted — the audit trail outlives the account.

> **Reading the delete rules.** `RESTRICT` dominates on purpose: almost nothing in this schema should be deletable while history references it. `CASCADE` appears only where the child has no independent meaning (a grant's locations, a template's sets). `SET NULL` appears where the reference is provenance rather than dependency — a session surviving its timetable rule, a booking surviving its class pack. If a new relationship seems to want `CASCADE` on something a member might later ask about, it wants `RESTRICT` and a soft delete instead.

16. Row-Level Security Application

Three policy shapes cover all 74 tables. Every table gets exactly one, in the same migration that creates it (`LB-ENG-1.0` §6).

Shape	Applies to	Rule
A — gym + location	The majority: `check_ins`, `bookings`, `sessions`, `memberships`, `payments`, `leads` …	Super Admin bypass, or matching `gym_id` and either gym-wide scope or a matching `location_id`.
B — gym only	Gym-wide entities: `plans`, `class_types`, `routines`, `message_templates`	Super Admin bypass, or matching `gym_id`. No location clause — these are gym-wide by definition (`FR-ORG-12`).
C — user-owned	Personal data: `fitness_profiles`, `body_metrics`, `workout_logs`, `health_screenings`	Own rows always; otherwise only if the caller administers or is assigned to that user's location. This is where the permission matrix of `LB-ENG-1.0` §13 is enforced at the data layer.

```sql
-- Shape A, the standard case
CREATE POLICY p_bookings_isolation ON bookings USING (
    current_setting('app.is_super_admin', true)::boolean IS TRUE
 OR (gym_id = current_setting('app.gym_id', true)::uuid
     AND (current_setting('app.gym_wide', true)::boolean IS TRUE
          OR location_id = ANY(string_to_array(
               current_setting('app.location_ids', true), ',')::uuid[]))
);

-- The exception: exercises, where NULL gym_id means the global catalogue.
-- Omitting the IS NULL branch yields an empty exercise library — an easy
-- mistake with a confusing symptom, so it has its own test.
CREATE POLICY p_exercises_isolation ON exercises USING (
    gym_id IS NULL
 OR current_setting('app.is_super_admin', true)::boolean IS TRUE
 OR gym_id = current_setting('app.gym_id', true)::uuid
);
```

Tables deliberately exempt from RLS. `users` (a person may belong to several gyms; scoping happens through `role_grants` and `member_records`), `subscription_offerings` and `content_pages` (platform-wide), `muscle_groups`, `equipment` and `foods` (global reference data), and `job_queue` (accessed only by workers using a dedicated role). Each exemption is a deliberate decision recorded in the migration that creates the table, never an omission — and the isolation test suite asserts the exempt list explicitly, so adding a table to it requires changing a test.

17. Indexing Strategy

Access pattern	Index	Why
RLS filtering — every query	`(gym_id, location_id)` on every scoped table	Load-bearing, not incidental: the policy predicate is evaluated per row, so without this every query degrades to a scan.
Attendance history, streaks	`(user_id, occurred_at DESC)`	The member's most-viewed screen.
Footfall reporting	`(location_id, occurred_at DESC)`	Admin dashboard, hourly and daily rollups.
Timetable browse	`(location_id, starts_at)` partial where not cancelled	Serves both the app and the public location page.
Booking lookups	`(session_id, state)`, `(user_id, booked_at DESC)`	Roster views and the member's own schedule.
Expiry sweeps	Partial on `(expires_on)` where state active	Keeps the nightly job cheap as history accumulates.
Job claiming	`(priority DESC, run_after)` partial where pending	What makes `SKIP LOCKED` polling inexpensive.
Workout history	`(user_id, performed_at DESC)`	Personal records derived by window function, not stored redundantly.
Metric charting	`(user_id, metric_type, recorded_on)`	Doubles as the daily uniqueness constraint.
Public slug lookup	`(slug)` unique on gyms, class_types, exercises, content_pages	Every public page render begins with a slug resolution.
"Gyms near me"	GiST on `locations(geo)`	Geofence validation and public proximity search, same index.
Lead abuse checks	`(ip_hash, created_at)`, `(phone_e164, created_at)`	Rate limiting on the one unauthenticated write path.

No partitioning at the SRS's stated volumes — 5 million attendance rows and 20 million workout sets are unremarkable for Postgres on modest hardware. Monthly range partitioning of `check_ins` and `workout_log_sets` is the first move if it ever becomes necessary, and nothing in this schema prevents adding it later.

18. Migration Sequence by Phase

Phase	Tables	Contents
V1	4	Extensions (`pgcrypto`, `citext`, `postgis`, `btree_gist`), the two database roles, all enum types, `media_assets`, `job_queue`, `idempotency_records`, `audit_log`.
V2	13	**Phase 1 — platform and identity.** `gyms`, `locations`, `location_hours`, `location_closures`, subscriptions and payments, `users`, `role_grants`, `role_grant_locations`, `refresh_tokens`, `user_consents`, `otp_attempts`.
V3	9	**Phase 1 — people.** `fitness_profiles` (with users, never after), `health_screenings`, `emergency_contacts`, `member_records`, `member_location_access`, trainer tables, `trainer_assignments`.
V4	11	**Phase 1 — billing.** Plans, memberships, freezes, payments, allocations, receipts, `receipt_sequences`, refunds. PT and class-pack tables created now, used in Phase 2.
V5	3	**Phase 1 — attendance.** `check_ins`, `location_qr_secrets`, `attendance_streaks`.
V6	5	**Phase 1 — engagement core.** `notifications`, `notification_preferences`, `device_tokens`, `message_templates`, `announcements`.
V7	14	**Phase 2 — training.** Exercise catalogue and seed data, workout templates, routines, logs, personal records, diet plans and consultations.
V8	6	**Phase 2 — progress and feedback.** `body_metrics`, `goals`, `measurement_reminders`, `feedback`, `feedback_responses`, `coaching_notes`.
V9	7	**Phase 3 — scheduling.** `class_types`, `rooms`, `timetable_entries`, `sessions`, `bookings`, `pt_appointments`, `session_ratings`.
V10	4	**Phase 3/4 — public surface.** `leads`, `lead_activities`, `content_pages`, `url_redirects`, plus SEO columns added to existing entities.
V11	4	**Phase 4 — insights and AI.** `ai_generations`, `report_snapshots`, `whatsapp_messages`, `foods` and `diet_adherence_logs`.

> **Two sequencing rules worth stating.** First, `fitness_profiles` is created in the same migration as `users`, never later — if the two can exist apart even briefly, code will be written that assumes a user might not have one, and `FR-PFP-01` starts leaking exceptions. Second, tables whose rows arrive in a later phase (PT packages in V4, class packs before scheduling exists) are still created early where they are referenced by foreign keys — it is far cheaper than adding the constraint afterwards against live data.

19. Enum Catalogue

Type	Values
`gym_state`	PENDING_VERIFICATION · VERIFIED · LIVE · LAPSED · SUSPENDED · CLOSED
`subscription_state`	PENDING · LIVE · GRACE · LAPSED · CANCELLED
`account_state`	PENDING_VERIFICATION · ACTIVE · SUSPENDED · DEACTIVATED
`role_type`	SUPER_ADMIN · GYM_OWNER · LOCATION_ADMIN · TRAINER · MEMBER
`membership_state`	PENDING · ACTIVE · FROZEN · EXPIRED · CANCELLED
`payment_method`	CASH · UPI · CARD · BANK_TRANSFER · CHEQUE
`checkin_method`	QR · GEOFENCE · MANUAL
`checkin_purpose`	SHIFT · PERSONAL_WORKOUT · BOTH
`session_state`	SCHEDULED · IN_PROGRESS · COMPLETED · CANCELLED
`booking_state`	BOOKED · WAITLISTED · CANCELLED · ATTENDED · NO_SHOW
`appointment_state`	SCHEDULED · COMPLETED · CANCELLED · NO_SHOW
`delivery_mode`	ON_SITE · ONLINE · HYBRID
`credit_reason`	PURCHASE · SESSION_COMPLETED · LATE_CANCEL · EXPIRY · ADJUSTMENT · REFUND
`metric_type`	WEIGHT_KG · BODY_FAT_PCT · CHEST_CM · WAIST_CM · HIP_CM · ARM_CM · THIGH_CM · NECK_CM
`record_type`	MAX_WEIGHT · MAX_REPS · MAX_VOLUME · BEST_TIME
`level`	BEGINNER · INTERMEDIATE · ADVANCED
`discipline`	STRENGTH · CARDIO · FUNCTIONAL · YOGA · MOBILITY · MEDITATION · DANCE
`meal_type`	BREAKFAST · MID_MORNING · LUNCH · SNACK · DINNER · POST_WORKOUT
`dietary_preference`	VEGETARIAN · VEGAN · EGGETARIAN · NON_VEGETARIAN · JAIN
`feedback_category`	FACILITY · EQUIPMENT · TRAINER · CLEANLINESS · BILLING · APPLICATION · OTHER
`lead_state`	NEW · CONTACTED · TRIAL_BOOKED · TRIAL_ATTENDED · CONVERTED · LOST
`lead_source`	LOCATION_PAGE · GYM_PAGE · CLASS_PAGE · BLOG · DIRECT · REFERRAL
`job_state`	PENDING · CLAIMED · SUCCEEDED · FAILED · DEAD
`ai_feature`	WORKOUT_PROGRESSION · DIET_DRAFT · PROGRESS_NARRATIVE · CHURN_RISK
`consent_type`	TERMS · PRIVACY · MARKETING · WHATSAPP

Twenty-five enum types. Each is a deliberate closed set, and extending one is a migration — which is the point (`DM-08`). Note that `consent_type` separates `MARKETING` and `WHATSAPP` from `TERMS` and `PRIVACY`: `NFR-PRV-06` requires marketing consent be held separately from the consent to operate the service, and one enum value is the cheapest possible way to make that impossible to conflate.

20. Appendix A — Deviations from Earlier Documents

Deviation	Earlier	Now, and why
Role grant location scope	LB-ARCH §5.2 sketched `location_ids uuid[]`	`role_grant_locations` junction table. An array cannot carry a foreign key, so it will eventually reference a deleted location; the junction gives referential integrity, an indexed reverse lookup, and `RESTRICT` protection when closing a location. The RLS context builds its CSV from this table once per request.
Booking uniqueness	LB-ARCH §5.2 used `UNIQUE NULLS NOT DISTINCT`	Partial unique index on live states. Clearer intent, works on any Postgres version, and correctly permits re-booking after cancellation.
No `members` table	Implied by the SRS's language	A member is a user with a `MEMBER` grant. `member_records` holds only the per-gym commercial facts. This is what lets staff train without a second account (§2.1).
Progress model	SRS M13 listed photographs	Photographs removed per your instruction. `body_metrics` is narrow rather than wide — see §10.
PostGIS dependency	Not mentioned in LB-ARCH	Added. Geofence distance and public proximity search both need real geography; approximating in application code is slower and wrong at the edges.
Web stack	LB-ARCH §2 specified React + Vite for the console	**Next.js**, one app with a public SSG/ISR zone and an authenticated client-rendered console. Required by the SEO decision; the console gets `noindex`.

21. Appendix B — Amendments Required to SRS and Architecture

Accumulating alongside the items already in LB-ARCH Appendix B and LB-ENG Appendix D. The SRS remains at v1.1 until you call for the v1.2 pass.

Document	Item	Amendment
LB-SRS-1.1 §10	"A public marketing website and lead management funnel" listed as out of scope	**Remove from Out of Scope.** Both are now in scope: a public directory on one domain, and lead capture from location pages. Add functional requirements for the public surface and the lead pipeline, and note that `leads` is the only unauthenticated write path in the system.
LB-SRS-1.1 M13	Progress photographs	Withdraw `FR–PRG–03` and the photograph elements of `FR–PRG–06` / `FR–PRG–07` (already logged in LB-ENG Appendix D).
LB-ARCH-1.0 §2	Web console stack	Change React + Vite to Next.js, one app, two zones. Note the `noindex` on authenticated routes and ISR revalidation for public pages.
LB-ARCH-1.0 ADR-03	Postgres extensions	Record `postgis` and `btree_gist` as required, with the constraints that depend on them (geofencing, proximity search, trainer overlap exclusion).
LB-ARCH-1.0 §5.2	Schema sketch	Superseded in full by this document. Retain the sketch for context, mark it illustrative.
LB-SRS-1.1 `OPN–04`	A person belonging to more than one gym	**Still open, and this schema now shows exactly what it costs.** `users` is deliberately gym-agnostic and `role_grants`, `member_records` and `trainer_profiles` are all per-gym, so the data model already supports it. What does not is the RLS context, which assumes one active `gym_id` per request. Resolving it means a gym-switching claim in the token — a contained change, but one that must be made before `identity` is built.

What to do with this document. It is the input to the Flyway migrations, and the migrations are the executable form of it — so when the two disagree, the migrations win and this document is corrected in the same pull request. The parts most worth re-reading before writing any code are §16 (every table needs its policy, in the same migration) and §15's note on delete rules, because both describe mistakes that are cheap now and very expensive once there is production data resting on them.

— END OF DOCUMENT —